

DOCUMENTATION TECHNIQUE COMPLÈTE

Projet Indicateurs

Microservice de suivi de performance e-learning pour PLaTon

BACKEND

NestJS · Port 3001

FRONTEND

Angular 21 · Port 4200

BASE DE DONNÉES

PostgreSQL × 2

PHILOSOPHIE

Zéro redéploiement

MOTEUR

DSL Pipeline JSON

DATE

Juin 2026

Table des matières

1. Vue d'ensemble et philosophie	→
2. Architecture technique	→
2.1 Structure des dossiers	→
2.2 Stack et dépendances	→
3. Bases de données	→
3.1 BDD PLaTon (lecture seule)	→
3.2 BDD Indicators (lecture/écriture)	→
4. Modèle de données — les 5 entités	→
5. Moteur DSL — les 10 étapes du pipeline	→
5.1 Fonctionnement général	→
5.2 Détail des 10 étapes	→
5.3 Sécurité du DSL	→
6. Modèle Option B+ — contextType + visualizations	→
7. Routes API complètes	→
7.1 Module Indicators	→
7.2 Module Ingestion	→
7.3 Module Préférences	→
7.4 Modules Cours et Ressources	→
8. Frontend Angular — toutes les pages	→
8.1 Dashboard & Vue d'ensemble	→
8.2 Wizard Administrateur	→
8.3 Carte indicateur & page détail	→
8.4 Page activité & Snapshots groupes	→
8.5 Onglets Cours et Ressources	→
9. Familles d'indicateurs et visibilité par rôle	→
10. Préférences utilisateur et personnalisation	→
11. Flux de données complets	→
12. Calcul incrémental et rafraîchissement du cache	→
13. État actuel — limites et stubs	→

1 Vue d'ensemble et philosophie

Le projet **Indicateurs** est un microservice autonome développé pour la plateforme pédagogique **PLaTon**. Son objectif : permettre à un administrateur de créer, configurer et publier des *indicateurs de performance e-learning* sans jamais toucher au code source ni redéployer l'application.

Philosophie fondatrice : Toute la logique de calcul est stockée en base de données sous forme d'un pipeline JSON interprété à la volée. Ajouter un nouvel indicateur revient uniquement à insérer une ligne dans une table.

Qu'est-ce qu'un indicateur ?

Un indicateur est une métrique pédagogique définie dynamiquement. Il répond à des questions comme :

- Quel est le taux de réussite moyen d'un étudiant sur une activité ?
- Combien de tentatives faut-il en moyenne aux étudiants d'un groupe pour réussir ?
- Quelle est la distribution des notes dans un cours ?
- Quel exercice pose le plus de difficultés ?

Piliers techniques



Zéro redéploiement

Les formules de calcul sont des pipelines JSON stockés en JSONB PostgreSQL et interprétés en mémoire à chaque exécution.

[JSONB](#) · [NestJS VM](#) · [TypeORM](#)



Multi-visualisations

Un indicateur peut avoir N visualisations (carte, jauge, graphe en ligne, histogramme, barres), chacune avec sa propre formule.

[Option B+](#) · [ECharts](#) · [ng-zorro](#)



Contextes multiples

6 contextes possibles : apprenant, enseignant, admin, cours, activité, groupe de TP. Chaque indicateur cible un seul contexte.

[ContextType](#) · [RBAC](#) · [RoleService](#)



Cache intelligent

Les valeurs calculées sont persistées en BDD pour éviter les recalculs. Cache-first avec invalidation sur ingestion d'événements.

[indicator_values](#) · [refreshActivityViews](#)

2

Architecture technique

2.1

Structure des dossiers

Backend — api/src/

```
app.module.ts
modules/
  core/
    config/
      configuration.ts      # ports, BDD, Redis, cron
    database/
      database.module.ts
      platon.datasource.ts  # PLATON_DATA_SOURCE (RO)
      indicators.datasource.ts # indicators (RW)
      guards/admin.guard.ts # STUB - toujours true
      platon/platon.service.ts # SQL brut sur PLaTon
    features/
      indicators/
        entities/          # 5 entités TypeORM
        interpreter/
          formula-interpreter.service.ts # DSL
          indicators.controller.ts
          indicators.service.ts
      ingestion/           # événements temps réel
      aggregation/         # cron daily 1h AM
      user-preferences/     # préférences utilisateur
      courses/             # API v1/courses
      resources/           # API v1/resources
      users/
      shared/
        constants/
        types/
        utils/date.utils.ts, math.utils.ts
```

Frontend — frontend/src/app/

```
core/
  models/indicator.model.ts
  services/
    indicator.service.ts
    dashboard-settings.service.ts
    role.service.ts
    user.service.ts
  guards/indicator.guard.ts
  interceptors/auth.interceptor.ts

features/
  dashboard/
    pages/
      overview/           # grille de cards
      indicators/         # préférences user
      widgets/sidebar,toolbar
    admin/
      indicator-detail/   # page détail + charts
      indicator-selector/ # activer/désactiver
      courses/            # onglet PLaTon porté
      resources/          # onglet PLaTon porté

shared/
  ui/
    indicator-card/       # card + modal config
  utils/
    indicator-family-grouping.ts

platon-stubs/            # stubs Angular 18-21
```

2.2

Stack et dépendances

Couche	Technologie	Version	Rôle
Backend	NestJS	^10	Framework API REST
Backend	TypeORM	^0.3	ORM, 2 DataSources
Backend	PostgreSQL	14+	Deux BDD distinctes
Backend	Node VM	intégré	Sandbox étape <code>js</code> du DSL
Frontend	Angular	21	SPA
Frontend	ng-zorro-antd	^17	Composants UI Ant Design
Frontend	Angular Material	^21	Icônes, drawer, tooltip
Frontend	ngx-echarts	^17	Graphiques (line, bar, gauge, histogram)
Frontend	RxJS	^7	Flux réactifs, BehaviorSubject

Contrainte d'isolation Angular : PLaTon utilise Angular 18, le microservice Angular 21. Les composants PLaTon ne peuvent pas être importés directement (double instance Angular). Solution : **stubs locaux** dans [src/platon-stubs/](#) avec des alias [tsconfig.json](#) .

3

Bases de données

3.1 BDD PLaTon — lecture seule (`PLATON_DATA_SOURCE`)

Cette base contient toutes les données pédagogiques de PLaTon. Le microservice ne l'écrit jamais — uniquement des `SELECT`.

Table	Colonnes clés	Usage dans les indicateurs
<code>SessionData</code>	<code>id, user_id, activity_id, resource_id, grade, attempts, created_at</code>	Table principale — source de 90% des indicateurs
<code>Sessions</code>	<code>id, activity_id, user_id</code>	Sessions de travail
<code>Activities</code>	<code>id, source (JSONB avec title), course_id</code>	Activités pédagogiques. Titre dans <code>source->'variables'->'title'</code> , fallback <code>Resources.name</code>
<code>Resources</code>	<code>id, name, type, code</code>	Ressources pédagogiques (exercices, etc.)
<code>Courses</code>	<code>id, name, desc, owner_id, is_test</code>	Cours. Pas de colonne <code>isActive</code> .
<code>Users</code>	<code>id, first_name, last_name, role</code>	Utilisateurs — résolution des noms
<code>CourseGroups</code>	<code>id (UUID), group_id (varchar), course_id, name</code>	Groupes de TP dans un cours
<code>CourseGroupsMember</code>	<code>group_id (varchar), user_id</code>	Membres d'un groupe de TP
<code>CourseMembers</code>	<code>course_id, user_id, role</code>	Membres d'un cours

Accès DSL dynamique : Toutes les tables du schéma `public` sont accessibles depuis le builder DSL. Validées via `information_schema` + regex anti-injection. Colonnes sensibles automatiquement masquées (password, token, email, hash, salt...).

3.2 BDD Indicators — lecture/écriture (`indicators`)

Base dédiée au microservice. Gérée avec `synchronize: true` en développement — les tables sont créées automatiquement au démarrage.

- `indicator_definitions`
- `indicator_values`
- `indicator_formula_versions`
- `indicator_execution_logs`
- `indicator_snapshots`
- `user_indicator_preferences`

4

Modèle de données — les entités

indicator_definitions — définition d'un indicateur

indicator_definitions

id	UUID PK	Identifiant unique
name	VARCHAR UNIQUE	Nom affiché
description	TEXT	Description libre
contextType	VARCHAR	learner teacher admin course activity group
familyName	VARCHAR 255	Famille d'indicateurs (nullable)
requiredEvents	JSONB	Événements déclencheurs
visualizations	JSONB	Tableau de N visualisations
formula	JSONB	Formule partagée (fallback)
isActive	BOOLEAN	Activé/désactivé
usageCount	INT	Nombre d'utilisateurs actifs

indicator_values — valeurs calculées

id	UUID PK	Identifiant
indicatorId	UUID FK	→ indicator_definitions
contextType	VARCHAR	learner, course, group...
contextId	VARCHAR	userId OU courseId:actId:vizId
value	DECIMAL	Valeur scalaire
structuredValue	JSONB	Valeur structurée (histogramme, barres...)
metadata	JSONB	Métadonnées optionnelles
UNIQUE		(indicatorId, contextType, contextId)

user_indicator_preferences

userId	VARCHAR	Utilisateur PLaTon
indicatorId	UUID FK	Indicateur concerné
isVisible	BOOLEAN	Affiché dans le dashboard
displayPreferences	JSONB	{ icon?, color? } par viz
activeVizId	VARCHAR	Viz active choisie par l'user
enabledVizIds	JSONB	Vizes visibles. null = toutes
UNIQUE		(userId, indicatorId)

indicator_snapshots — comparaison groupes

id	UUID PK	Identifiant
indicatorId	UUID FK	→ indicator_definitions
contextType	VARCHAR	default 'group'
contextId	VARCHAR	groupId épinglé
activityId	VARCHAR	Activité concernée
title	VARCHAR	Titre affiché sur le snapshot
UNIQUE		(indicatorId, contextType, contextId, activityId)

indicator_formula_versions — historique

id	UUID PK	
indicatorId	UUID FK	
formula	JSONB	Snapshot de la formule
createdAt	TIMESTAMP	Date de sauvegarde

indicator_execution_logs

indicatorId	UUID FK	
userId	VARCHAR	userId ou groupId
value	DECIMAL	Valeur scalaire calculée
durationMs	INT	Durée d'exécution en ms
error	TEXT	Message d'erreur si échec

Clé composite contextId pour course/group/activity

Pour les contextes **course**, **group** et **activity**, la clé `contextId` dans `indicator_values` est composite afin de différencier le même groupe vu depuis deux activités différentes, ou la même vue calculée avec deux visualisations différentes :

```
contextId = "{originalContextId}:{activityId}:{vizId}"  
  
Exemple : "grp-abc123:act-xyz789:viz-001def"
```

Pour les apprenants (`contextType: 'learner'`), `contextId` reste simplement le `userId` .

5 Moteur DSL — les 10 étapes du pipeline

5.1 Fonctionnement général

Le moteur DSL ([FormulaInterpreterService](#)) exécute une suite d'étapes en cascade. La sortie d'une étape devient l'entrée de la suivante. Le résultat final peut être :

number → card / gauge / line-chart

object { clé: valeur } → bar-chart

array [{ bucket, count }] → histogram

Le contexte d'exécution ([FormulaContext](#)) injecte automatiquement les identifiants de l'utilisateur courant dans les étapes qui en ont besoin :

```
interface FormulaContext {
  userId?: string; // pour contexte learner
  activityId?: string; // pour contexte activity / group
  courseId?: string; // pour contexte course
  groupId?: string; // pour contexte group
  indicatorId?: string; // pour le log d'exécution
}
```

En cas d'erreur sur une étape, le pipeline s'arrête, log l'erreur dans [indicator_execution_logs](#), et retourne 0 (pas une exception fatale).

5.2 Détail des 10 étapes

1

fetch — Récupération de données

Charge un ensemble de lignes depuis une table PLaTon en filtrant par le contexte courant. **Toujours la première étape.**

params : { table: "SessionData" | toute table PLaTon, contextFields: ["user_id", "activity_id", "course_id", "group_id"] }

Cas spécial [group_id](#) : déclenche une jointure automatique via [CourseGroupsMember](#) pour ramener les sessions de tous les membres du groupe. Si [activityId](#) est absent → retourne [] immédiatement (guard explicite).

2

join — Jointure avec une seconde table

Fusionne les lignes en entrée avec une table PLaTon droite. Indexe la table droite par [rightKey](#) en [Map](#) (O(n) au lieu de O(n²)).

params : { table, leftKey, rightKey, contextFields?, joinType?: "left"(défaut) | "inner" | "right" | "full" }

En cas de conflit de nom de colonne, les champs **gauches sont prioritaires**. Les lignes sans correspondance sont conservées en LEFT JOIN.

3

filter — Filtre des lignes

Sélectionne uniquement les lignes satisfaisant une condition sur un champ.

params : { field, operator: "==" | "!=" | ">" | "<" | ">=" | "<=", value }

Exemple : garder uniquement les sessions avec [grade >= 50](#)

4

groupBy — Regroupement

Regroupe les lignes par la valeur d'un champ. Retourne un tableau de groupes ([any\[\]\[\]](#)) au lieu d'un tableau plat.

params : { groupField: "resource_id" | "user_id" | ... }

Utile pour calculer une statistique par exercice ou par étudiant avant d'agréger.

5

findFirst — Sélection d'une ligne par groupe

Accepte des groupes (`any[][]`) ou un tableau plat. Sélectionne une ligne par groupe selon un critère optionnel.

params : { whereField?, whereValue?, sortField? }

Exemple : extraire la meilleure tentative (`sortField: "grade"`) de chaque exercice.

6

extract — Extraction d'un champ numérique

Extrait un champ de chaque ligne et retourne un tableau de nombres. Convertit via `parseFloat` . Les NaN sont filtrés.

params : { extractField: "grade" | "attempts" | ... }

7

aggregate — Agrégation en valeur scalaire

Réduit un tableau de nombres en une seule valeur.

params : { aggregateFn: "avg" | "sum" | "count" | "min" | "max" }

Retourne `0` si le tableau est vide.

8

round — Arrondi

Arrondit une valeur numérique à N décimales.

params : { decimals: 2 } (défaut : 2)

9

divide — Division

Divise la valeur courante par un diviseur constant. Division par zéro retourne `0` .

params : { divideBy: 100 } (ex: convertir en pourcentage)

10

js — Code JavaScript arbitraire (sandbox)

Exécute du code JS dans un sandbox Node.js (`vm.createContext`) avec timeout de 2 secondes. La variable `input` contient la sortie de l'étape précédente. Le code doit assigner `result` .

params : { code: "result = input.filter(x => x > 0).length;" }

⚠️ Sandbox Node.js léger (`vm`) — suffisant pour un usage interne. Pour une production exposée, remplacer par `isolated-vm` .

5.3 Sécurité et protection du DSL

Mécanisme	Portée	Détail
Validation des noms de tables	<code>queryTable()</code>	Interroge <code>information_schema.columns</code> + regex anti-injection SQL. Toute table inconnue ou nom invalide → erreur immédiate.
Masquage des colonnes sensibles	<code>PlatonService</code>	Pattern automatique : <code>/password passwd secret token api_key hash salt credential email phone discord ip_address/i</code> . S'applique à toutes les tables, même futures.
Sandbox JS	Étape <code>js</code>	<code>vm.createContext()</code> + timeout 2s. Messages d'erreur nettoyés avant renvoi client (chemins masqués).
Cache colonnes sûres	<code>safeColumnsCache</code>	Map en mémoire évitant de requêter <code>information_schema</code> à chaque appel DSL.

5.4 Mode debug — `preview-steps`

L'endpoint `POST /api/indicators/preview-steps` exécute le pipeline via `interpretWithSteps()` et retourne le snapshot intermédiaire de chaque étape avec sa durée en millisecondes — utile pour déboguer une formule complexe.

6

Modèle Option B+ — contextType + visualizations[]

L'**Option B+** est le modèle architectural central du projet. Il est implémenté depuis mi-2026 et remplace tous les anciens modèles multi-contexte.

Règle fondamentale : 1 indicateur = 1 `contextType` unique = N `visualizations[]` indépendantes, chacune avec sa propre formule DSL et ses propres seuils.

Structure d'une visualisation

```
interface IndicatorVisualization {
  id: string; // UUID stable généré au moment de la création
  label: string; // Label de l'onglet (ex: "Réussite", "Distribution")
  type: VizType; // 'card' | 'gauge' | 'line-chart' | 'bar-chart' | 'histogram'
  icon?: string; // Icône Material Design (ex: "trending_up")
  color?: string; // Couleur hex personnalisée
  unit?: string; // Unité affichée (ex: "%", "pts", "min")
  thresholds?: { // Seuils de performance par visualisation
    good: number;
    warning: number;
    danger: number;
  };
  formula?: FormulaDefinition | null; // Formule propre à cette viz
  // Si null → fallback sur indicator.formula
}
```

Les 6 contextTypes

contextType	Qui voit	contextId passé	activityId requis
learner	Étudiant	userId	Non (TARGET_ACTIVITY_ID via env)
teacher	Enseignant	teacherId	Non
admin	Admin	adminId	Non
course	Tous rôles	courseId	Oui (sélecteur teacher)
activity	Tous rôles	activityId	Non (contextId = activityId)
group	Enseignant, Admin	groupId	Oui (sélecteur teacher)

Les 5 types de visualisation

Type	Rendu	Retour DSL attendu	Cas d'usage
card	Valeur numérique + icône	number	Score moyen, nombre de tentatives
gauge	Jauge ECharts 0–100	number	Taux de réussite en %
line-chart	Évolution temporelle	number	Progression dans le temps
bar-chart	Barres horizontales	object { clé: valeur }	Résultats par exercice
histogram	Distribution avec tooltip users	array [{ bucket, count, users? }]	Distribution des notes

Logique de fallback de la formule



Clé de cache `contextId` composite

Pour les contextes course, group, activity, la clé de cache dans `indicator_values` inclut l'activityId et le vizId pour éviter les collisions :

```
contextId = `${originalContextId}:${activityId}:${viz.id}`
```

Le système vérifie d'abord si cette clé existe → retourne directement si oui (cache-first). Calcule et stocke uniquement si la combinaison n'existe pas encore (compute-once).

7

Routes API complètes

Toutes les routes sont **publiques** (pas d'authentification JWT active). Le guard admin est un stub qui retourne toujours `true`. Préfixe global : `/api`.

7.1 Module Indicators — `/api/indicators`

⚠ Les routes littérales (`all` , `schema` , `dashboard` , `preview` , `preview-steps` , `precompute-context`) doivent impérativement être déclarées **avant** `/:id` dans le contrôleur.

Méthode	Route	Description
GET	/indicators	Tous les indicateurs actifs
GET	/indicators/all	Tous (actifs + inactifs) — pour l'admin
GET	/indicators/schema	Tables PLaTon + colonnes disponibles (builder)
GET	/indicators/teacher/:id/context	Cours + groupes d'un enseignant
GET	/indicators/course/:id/activities	Activités d'un cours
GET	/indicators/course/:id/students	Étudiants d'un cours (test de formule)
GET	/indicators/:id	Détail d'un indicateur
GET	/indicators/:id/context-configs	Alias compat → retourne { contextType, visualizations, formula }
GET	/indicators/:id/values? contextType=&contextId=&period=&limit=	Historique de valeurs
GET	/indicators/:id/usage	Compteur d'utilisateurs actifs
GET	/indicators/:id/formula-history	Versions successives de la formule
GET	/indicators/:id/logs?limit=	Logs d'exécution du pipeline
GET	/indicators/:id/snapshots?activityId=	Snapshots groupes épinglés
POST	/indicators	Créer un indicateur
POST	/indicators/dashboard	Valeurs en batch pour le dashboard
POST	/indicators/preview	Tester une formule sans persister → { result }
POST	/indicators/preview-steps	Debug pas-à-pas de la formule → { steps[] }
POST	/indicators/:id/compute-view	Calculer + cacher une vue pour un contexte donné
POST	/indicators/:id/recalculate	Recalcul forcé (invalidation du cache)
POST	/indicators/:id/rollback/:versionId	Restaurer une version antérieure de la formule
POST	/indicators/:id/snapshots	Créer un snapshot groupe (409 si doublon)
PATCH	/indicators/:id	Modifier un indicateur
PATCH	/indicators/:id/status	Activer/désactiver
PATCH	/indicators/:id/snapshots/:snapshotId	Modifier le titre d'un snapshot
DELETE	/indicators/:id	Supprimer un indicateur
DELETE	/indicators/:id/snapshots/:snapshotId	Supprimer un snapshot

7.2 Module Ingestion — [/api/ingest](#)

Méthode	Route	Description
POST	/ingest	Ingérer un événement PLaTon (réponse 202 immédiate, traitement async)
POST	/ingest/batch	Ingérer N événements en une requête

Format d'un événement : { type, userId, courseId?, activityId?, exerciseId?, timestamp, payload, metadata? }
Après traitement, déclenche automatiquement `refreshSnapshots` + `refreshActivityViews` en fire-and-forget.

⚠️ Aucun producteur actif : L'endpoint existe et fonctionne, mais PLaTon n'envoie pas encore d'événements vers ce microservice. Le calcul incrémental est donc opérationnel techniquement mais jamais déclenché en pratique.

7.3 Module Préférences — [/api/preferences](#)

Méthode	Route	Body / Query	Description
GET	/preferences?userId=	—	Préférences d'un utilisateur → { activeIndicators[], vizPreferences, vizVisibility }
PATCH	/preferences/:indicatorId?userId=	{ isVisible?, activeVizId?, enabledVizIds?, userRole? }	Mise à jour des préférences. Si <code>userRole === 'teacher' 'admin'</code> → skip précalcul learner.

7.4 Modules Cours et Ressources — /api/v1

Méthode	Route	Description
GET	/v1/courses	Recherche avec filtres (search, period, offset, limit, order)
GET	/v1/courses/:id	Cours avec statistiques (studentCount, teacherCount, activityCount)
GET	/v1/courses/:id/sections	Sections ordonnées
GET	/v1/courses/:id/activities	Activités (avec titre JSONB fallback, état, progression)
GET	/v1/courses/:id/groups	Groupes de TP du cours
GET	/v1/courses/:cId/groups/:gId/members	Membres d'un groupe avec rôle et date
GET	/v1/courses/:cId/activities/:aId	Détail d'une activité
GET	/v1/courses/_/activities/:aId/results	Résultats d'une activité (stats + par utilisateur)
GET	/v1/courses/_/activities/:aId/results/date	Résultats sur une plage de dates
GET	/v1/courses/_/activities/:aId/csv	Export CSV des résultats
GET	/v1/resources	Recherche ressources (types, status, owners, parents...)
GET	/v1/resources/tree	Arbre de cercles hiérarchique
GET	/v1/resources/completion	Autocomplétion des noms
GET	/v1/resources/owners	Propriétaires distincts
GET	/v1/resources/user-circle?userId=	Cercle personnel d'un utilisateur
GET	/v1/resources/:id	Ressource par id ou code

7.5 Modules Groupes et Utilisateurs

Méthode	Route	Description
GET	/groups?teacherId=	Groupes de TP d'un enseignant
GET	/groups/members?groupId=	Membres d'un groupe de TP
GET	/users/:id	Informations d'un utilisateur PLATon

8 Frontend Angular — toutes les pages et composants

8.1 Routing global

```

/                → redirect /dashboard
/dashboard       → DashboardPage (shell sidebar + toolbar)
/overview        → OverviewPage (grille de cards)
/indicators      → IndicatorsPage (sélecteur user)
/indicator/:id   → IndicatorDetailComponent
/admin           → AdminIndicatorManagerComponent
/courses         → CoursesPage (PLaTon porté)
  /:id           → CoursePage
    /dashboard   → CourseDashboardPage
    /activities/:aId → ActivityPage (statistiques + indicateurs)
    /members     → CourseMembersPage
    /students    → CourseStudentsPage
    /teachers    → CourseTeachersPage
    /groups      → CourseGroupsPage
    /settings    → CourseSettingsPage
/resources       → ResourcesPage (PLaTon porté)
  /:id           → ResourcePage
    /overview    → ResourceOverviewPage
    /browse      → ResourceBrowsePage
    /events      → ResourceEventsPage
    /settings    → ResourceSettingsPage
  /informations
  /members
  /template

```

8.2 Sidebar et navigation

La sidebar (`SideBarComponent`) charge l'utilisateur courant au démarrage via `UserService.getUserById(environment.defaultUserId)` et appelle `RoleService.setRole(user.role)` . Liens de navigation :

 Tableau de bord → /overview

 Mes indicateurs → /indicators

 Cours → /courses

 Espace de travail → /resources

 Admin → /admin (si admin)

Changement de rôle pour les tests : Modifier `defaultUserId` dans `environment.ts` . Les UUIDs disponibles sont commentés sur la même ligne : `student` , `admin` (valeur actuelle), `teacher` . Le `localStorage` ne fonctionne pas car la sidebar écrase systématiquement le rôle au chargement.

8.3 OverviewPage — tableau de bord principal

Affiche la grille d' `IndicatorCard` pour tous les indicateurs activés par l'utilisateur. Comportements clés :

- Filtre les indicateurs par `contextType === scope + canSeeIndicatorContext()`
- Pour les enseignants : affiche `TeacherContextSelectorComponent` (sélecteur Cours → Activité → Voir par)
- Restaure le contexte teacher sauvegardé au `ngOnInit` → cards affichent les valeurs sans recalcul
- Déclenche `precomputeContext` dès qu'une activité est sélectionnée (fire-and-forget)

8.4 IndicatorCardComponent — la carte indicateur

Composant central affiché dans le dashboard. Logique de chargement :

Scope	Action	Endpoint appelé
<code>learner</code>	Lecture valeur pré-calculée	<code>GET /values?contextType=learner&contextId=userId</code>
<code>course group activity</code>	Compute-view (cache-first)	<code>POST /:id/compute-view</code>

Fonctionnalités de la card :

- Bouton crayon (toujours visible) → ouvre `IndicatorConfigModalComponent`
- Visualisation active (`activeVizId`) choisie parmi les `visibleVisualizations`
- Couleur des seuils selon `getThresholdColor()` de la viz active
- Navigation vers page détail via `[routerLink]` toujours actif
- Rechargement automatique via `ngOnChanges()` quand le contexte change

Modal d'édition — `IndicatorConfigModalComponent`

Ouvert depuis le bouton crayon de la card. Permet :

- **Sélection des visualisations visibles** : checkboxes par viz, minimum 1 obligatoire (la dernière cochée est désactivée)
- La sélection est persistée en BDD dans `user_indicator_preferences.enabled_viz_ids`
- Flux : Card → `ModalDataService.setData()` → `NzModalService.create()` → Modal lit le service

8.5 IndicatorDetailComponent — page de détail

Accessible depuis `/dashboard/indicator/:id` + query params de contexte.

Query param <code>from</code>	Contexte	Bannière affichée
<code>(absent)</code>	Learner (userId par défaut)	Aucune
<code>activity</code>	Activité via activityId	Cours → Activité + lien retour
<code>group-snapshot</code>	Groupe via groupId + activityId	Cours → Activité → Groupe + lien retour
<code>(teacher)</code>	Cours/groupe depuis DashboardSettingsService	Bannière read-only cours + activité + scope

Fonctionnalités :

- Onglets `nz-tabs` par visualisation — calcul lazy à la demande
- Graphiques ECharts : bar-chart (labels tronqués 25 chars + tooltip complet), histogram (tooltip users résolus), line-chart (filtres période)
- Filtres période : `nz-radio-group` natif Ant Design → 7j / 30j / 90j / Tout / Personnalisé
- Plage personnalisée : `nz-range-picker` avec bornes inclusives 00:00–23:59:59
- Historique des versions de formule
- Logs d'exécution du pipeline

8.6 Wizard Administrateur — `IndicatorBuilderComponent`

Wizard en 3 étapes pour créer ou modifier un indicateur :

Étape	Titre	Contenu
1	Définition	Nom, description, événements déclencheurs, <code>contextType</code>
2	Vues	Liste de <code>FlatViz</code> — chaque vue : label, type (5 choix), icône (picker grille 37 icônes), couleur, unité, seuils Bon/Moyen/Critique
3	Formule	Pipeline DSL par vue — sélecteur onglets-boutons pour naviguer entre vues. Toutes les étapes DSL configurables.

Chaque étape du pipeline DSL dans le wizard expose des infobulles (`mat-icon info_outline` avec `nz-tooltip`) sur tous les champs — 25 icônes d'aide réparties sur les 3 étapes.

8.7 Page statistiques d'activité — `ActivityPage`

Route : `/dashboard/courses/:id/activities/:activityId`

- Statistiques ECharts : taux de réussite, score moyen, distribution des résultats
- **Section "Indicateurs"** : grille de cards pour les indicateurs `contextType === 'activity'`
- **Section "Par groupe"** : `GroupSnapshotsPanelComponent` si au moins un indicateur `group` existe

8.8 GroupSnapshotsPanelComponent — comparaison groupes

Permet d'épingler des groupes de TP côte à côte pour comparer leurs indicateurs sur une même activité.

- Bouton "+ Ajouter un groupe" → dropdown des groupes non encore épinglés
- Anti-doublon double couche : filtre frontend (masque les groupes déjà ajoutés) + 409 backend
- Titre éditable inline (PATCH) / supprimable (DELETE + popconfirm)
- Snapshots "vivants" : se mettent à jour automatiquement via `refreshSnapshots` à chaque ingestion

9

Familles d'indicateurs et visibilité par rôle

Familles d'indicateurs

Plusieurs indicateurs peuvent partager un même `familyName` pour être regroupés dans l'interface. Une famille est un regroupement purement nominal — aucun lien de calcul entre membres.

Helper partagé `buildIndicatorDisplayRows()` dans `shared/utils/indicator-family-grouping.ts` — fonction pure utilisée dans les deux listes (sélecteur user + manager admin). Retourne des `IndicatorDisplayRow` de type `'standalone' | 'family' | 'member'`.

La ligne `'family'` est une en-tête repliable (fond violet `#f9f0ff`, icône `folder_special`, chevron, badge nombre de membres). Les membres apparaissent indentés dessous si la famille est déployée.

Filtres de regroupement (dans les deux listes)

Tout — tous les indicateurs

Familles seulement

Sans famille seulement

Visibilité par rôle — `RoleService.canSeeIndicatorContext()`

Règle centralisée dans `role.service.ts` (table `INDICATOR_VISIBILITY`) :

contextType	student	teacher	admin	demo
learner	✓	✗	✗	✓
teacher	✗	✓	✗	✗
admin	✗	✗	✓	✗
course	✓	✓	✓	✓
activity	✓	✓	✓	✓
group	✗	✓	✓	✗

Cette règle est appliquée dans :

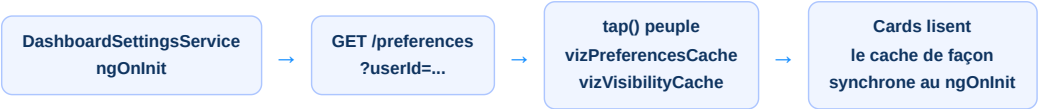
- `overview.page.ts` — filtre au chargement et au changement de contexte
- `indicator-selector.component.ts` — avant le filtre de scope (point critique)
- `activity.page.ts` — filtre les indicateurs `activity` ET `group`
- `admin-indicator-manager.component.ts` — N'applique PAS ce filtre (outil de gestion)

10 Préférences utilisateur et personnalisation

Ce que l'utilisateur peut personnaliser

Personnalisation	Stockage	API	Cache frontend
Activer/désactiver un indicateur	<code>is_visible</code> BDD	PATCH <code>/preferences/:id</code>	<code>DashboardSettingsService</code>
Visualisation active (onglet affiché)	<code>active_viz_id</code> BDD	PATCH <code>/preferences/:id</code>	<code>IndicatorService.vizPreferencesCache</code>
Visualisations visibles (multi-viz)	<code>enabled_viz_ids</code> JSONB BDD	PATCH <code>/preferences/:id</code>	<code>IndicatorService.vizVisibilityCache</code>
Couleur / icône par viz	<code>display_preferences</code> JSONB BDD	PATCH <code>/preferences/:id</code>	<code>IndicatorCardComponent.loadUserColorPreference()</code>

Flux de chargement des préférences au démarrage



Logique `visibleVisualizations`

```
get visibleVisualizations(): IndicatorVisualization[] {
  const all = indicator.visualizations ?? [];
  const visible = all.filter(v => indicatorService.isVizEnabled(indicator.id, v.id));
  return visible.length ? visible : all; // fallback : tout afficher
}
```

Persistence contexte enseignant entre navigations

`DashboardSettingsService` expose `TeacherSelectionState` :

- Sauvegardé à chaque changement dans `TeacherContextSelectorComponent` (avec les noms lisibles)
- Restauré au `ngOnInit` d' `OverviewPage` → les cards affichent les valeurs cachées immédiatement
- Relu dans `IndicatorDetailComponent` → bannière lecture seule sans re-sélection

11 Flux de données complets

Flux 1 — Étudiant active un indicateur

1

PATCH /preferences/:indicatorId?userId=...

Body : { isVisible: true }. Le serveur détecte que l'indicateur a un contextType 'learner'.

2

calculateAndStoreValue()

Récupère la première viz learner, exécute le pipeline DSL avec contexte userId = TARGET_ACTIVITY_ID, stocke dans indicator_values.

3

Dashboard rechargé

La card lit GET /values?contextType=learner&contextId=userId → retourne la valeur pré-calculée.

Flux 2 — Enseignant sélectionne cours + activité

1

TeacherContextSelectorComponent — emitContext()

Émet DashboardContext { scope, scopeId, activityId } + POST /indicators/precompute-context (fire-and-forget)

2

precomputeContext() côté serveur

Pour chaque indicateur actif : computeView(course, courseId, activityId) pour chaque visualisation. Cache intégré → pas de double calcul.

3

Cards chargent

POST /compute-view → si la valeur existe en BDD → retourne directement. Pas de recalcul DSL.

Flux 3 — Ingestion d'un événement PLaTon (théorique)

1

POST /ingest — IngestionController

Réponse 202 immédiate. Traitement asynchrone en arrière-plan via ingestEvent().

2

processIndicatorUpdate()

Pour chaque indicateur qui écoute l'événement : recalcule la valeur learner de l'utilisateur concerné. Met à jour indicator_values[learner, userId].

3

refreshSnapshots() — fire-and-forget

Recalcule tous les snapshots épinglés pour cette activité avec forceRefresh=true.

4

refreshActivityViews() — fire-and-forget

Recalcule toutes les indicator_values course/group/activity dont le contextId composite référence cette activityId.

12 Calcul incrémental et rafraîchissement du cache

Éligibilité au calcul incrémental

Une formule est "à forme incrémentale" si elle est de la forme :



Les formules avec `filter`, `join`, `groupBy` ou `js` sont **non éligibles** — elles sont recalculées intégralement à chaque événement.

Table de fidélité des valeurs en cache

Type de valeur	Mis à jour quand	Mécanisme
Learner (pré-calculé)	À chaque ingestion pour cet user	<code>processIndicatorUpdate()</code>
Snapshot épinglé	À chaque ingestion pour cette activité	<code>refreshSnapshots(forceRefresh=true)</code>
Cache course/group/activity consulté	À chaque ingestion pour cette activité	<code>refreshActivityViews(forceRefresh=true)</code>

Tradeoff fan-out : Chaque événement d'ingestion déclenche le recalcul de *toutes* les vues course/group/activity déjà consultées pour cette activité, pour tous les indicateurs concernés. En production avec beaucoup de groupes, cela peut représenter de nombreux recalculs DSL en arrière-plan par réponse d'étudiant. Aucune limite ni debounce n'est implémentée pour l'instant.

`forceRefresh` dans `computeView`

```
async computeView(  
  indicatorId, contextType, contextId, activityId?, vizId?,  
  forceRefresh = false // si true -> ignore le cache, recalcule et écrase en BDD  
)
```

En mode normal (`forceRefresh=false`) : vérifie d'abord la BDD, retourne si trouvé. En mode forcé : saute la lecture de cache, exécute le DSL, écrase la ligne via `upsert` .

13

État actuel — limites, stubs et reste à faire

Fonctionnalités complètes

Fonctionnalité	Statut
Wizard de création d'indicateurs (3 étapes)	 Complet
Moteur DSL (10 étapes)	 Complet
Option B+ (1 indicateur = N visualisations)	 Complet
Préférences utilisateur (viz active, viz visibles, couleur)	 Complet
Familles d'indicateurs + regroupement repliable	 Complet
Snapshots groupes vivants	 Complet
Cache course/group/activity avec invalidation	 Complet
Onglet Cours (PLaTon porté)	 Complet
Onglet Ressources (PLaTon porté, tous composants)	 Complet
Filtres période indicateur (nz-radio-group + date range)	 Complet
Page statistiques d'activité	 Complet
Sélecteur contexte enseignant (cours → activité → groupe)	 Complet
Visibilité par rôle centralisée	 Complet

Limites connues et stubs actifs

Élément	État	Impact
Authentification JWT	Stub vide	Toutes les routes sont publiques. <code>authInterceptor</code> n'injecte aucun token.
Guard admin	Toujours true	Toute personne peut accéder aux routes admin.
Ingestion d'événements	Pas de producteur	Le calcul incrémental fonctionne techniquement mais n'est jamais déclenché (PLaTon n'envoie pas d'événements).
Utilisateur courant	Hardcodé env	<code>environment.defaultUserId</code> — pas de vraie session. Changer l'UUID pour tester différents rôles.
Redis	Configuré non utilisé	Le cache est exclusivement PostgreSQL (BDD indicators).
Color picker dans le modal	UI manquante	La structure backend est prête (<code>display_preferences</code>), le sélecteur de couleur n'est pas encore visible dans le modal d'édition.
Sandbox JS (étape <code>js</code>)	vm léger	Node.js <code>vm</code> suffit pour un usage interne. Production exposée → migrer vers <code>isolated-vm</code> .
Opérations d'écriture PLaTon (cours, ressources)	Stubs vides	La BDD PLaTon est en lecture seule. Les boutons d'édition de cours/ressources ne font rien.

Comment démarrer le projet

```
# Backend (port 3001) — mode développement recommandé
cd indicateurs/api
nest start --watch

# Frontend (port 4200)
cd indicateurs/frontend
ng serve

# Changer de rôle utilisateur — modifier environment.ts
# student : e901cddd-0e08-4a3d-8aad-4d2c49f39fdd
# admin   : 055dc07c-3f4a-41d8-8d2d-828b75d441b4 (valeur actuelle)
# teacher : ee225671-e8b5-422d-9191-7189a5be58c8
```

Documents complémentaires

Fichier	Contenu
indicateurs/readme.md	Vue d'ensemble complète avec tous les \$
indicateurs/guide.md	Guide pas-à-pas pour créer chaque type d'indicateur
indicateurs/structure.md	Arborescence détaillée de tous les fichiers
indicateurs/parcours-donnees.md	Trace fichier-par-fichier de chaque route (composant → service → contrôleur → BDD)

Projet Indicateurs — Documentation Technique Complète

Microservice PLaTon · NestJS + Angular 21 · PostgreSQL × 2 · Moteur DSL Pipeline JSON

Généré le 16 juin 2026